

要約

- 本研究では、ニューラルネットワークの中間表現 (latent) を、単なる通過点ではなく、入力と出力の関係を表す大事なつなぎ目として考え直すことを目指しています。普通の end-to-end 学習では、入力から出力へ一方通行の変換が行われ、中間の latent はあまり意味を持たない特徴として使われることが多いです。そこで今回は、入力から出力への流れ ($x \rightarrow z \rightarrow y$) だけでなく、その逆 ($y \rightarrow z \rightarrow x$) の経路も作ることで、latent を両方向をつなぐ構造として捉え直す方法を試しました。そのために、通常の encoder・decoder に加えて、逆方向の encoder (inverse encoder) も取り入れ、 x と y の両方から latent にマッピングできるかどうかを検証しました。
- **データセット** : MNIST
- **モデル** : 自作の軽量版 ViT、U-Net、MLP、PCA+MLP
- **評価方法** : flow matching による vector field の再現誤差と類似度指標

目的と背景

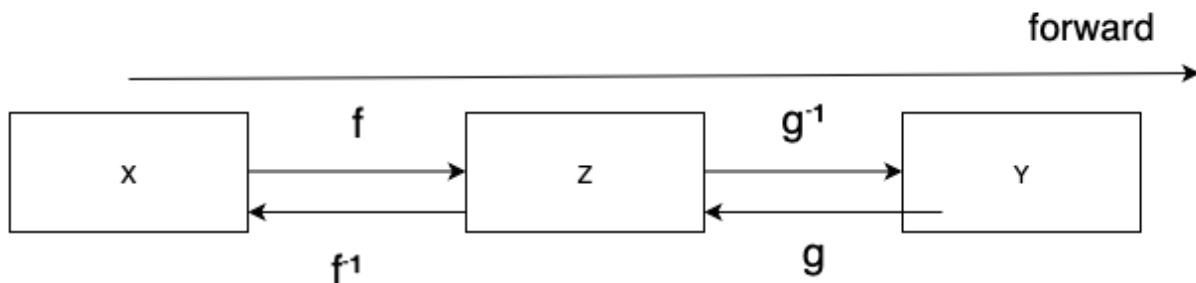
- 現在のニューラルネットワークは、入力 x から出力 y への変換を、ひとつの大きな関数として end-to-end で学習するのが一般的です。しかしこの方法では、その途中にある「中間表現 (latent)」がどんな意味を持っているのか、またそれがどれだけ解釈できるのかを理解するのは簡単ではありません。
- そこで本研究では、学習の流れを「経路 (path)」として捉え直し、 $x \rightarrow z \rightarrow y$ という順方向の流れだけでなく、逆方向の $y \rightarrow z \rightarrow x$ という経路もあわせて構築することを試みました。これによって、latent は単なる通過点ではなく、 x と y をつなぐ“構造的な橋渡し”としての役割を持つと考えられます。
- 本研究の目的は、こうした逆方向の情報を活用することで、より意味のある decoder を作ること、そして encoder-decoder の中で latent がどのような役割を果たすのかを明らかにすることです。また、逆方向の構造 (inverse encoder) から decoder を学習できるかどうかについても検討しました。それから x と y のマッピング関係を研究すること。

理論的視点と抽象構造

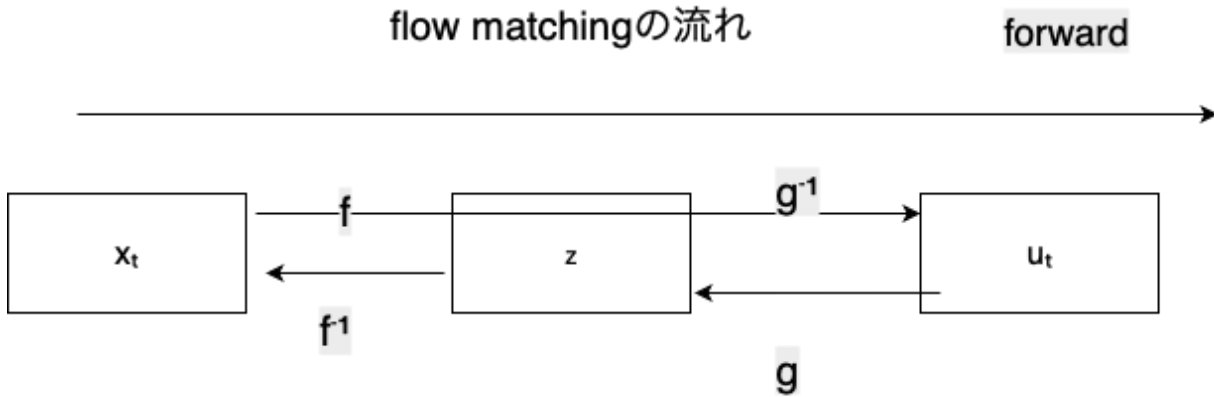
- ニューラルネットワークでは、「完全にラベルのない純粋なunsupervised学習」というものは実際には存在せず、何らかの形で正解（ground truth）に基づいた最適化が行われています。多くのモデルでは、入力 x から出力 y への一方方向の end-to-end 学習が基本となっており、その中で得られる latent（中間表現）は単なる特徴ベクトルとして扱われることが一般的です。
- ここで、もし $x \rightarrow y$ の変換を「情報を圧縮してエントロピー（不確かさ）が減る過程」と考えるなら、その逆の $y \rightarrow x$ は「情報を広げてエントロピーが増える生成の過程」として見ることができます。
- さらに数学的な観点からは、 x の確率的な分布が latent z を通じて y に変換されることを「測度変換（measure transformation）」としてとらえることもできます。そして、この変換がもし理論的に可逆であるなら、逆方向の推論や再構成も可能になるということです。
- 本研究では、このような理論的な背景をふまえ、順方向の $x \rightarrow z \rightarrow y$ と逆方向の $y \rightarrow z \rightarrow x$ の2つの経路を設計しました。これにより、 x と y の関係を latent を通じた「双方向的な構造」として再定義し、latent を単なる通過点ではなく、両者をつなぐ“構造的な変数”として捉え直すことを目指しています。

考えを図式にする

普通のニューラルネットワークの流れ



flow matchingの流れ



本研究では、入力と出力のあいだの「経路 (path)」に注目します。具体的には：

$x \rightarrow z$ 、 $z \rightarrow x$ ($= p(z|x), p(x|z)$)

$y \rightarrow z$ 、 $z \rightarrow y$ ($= p(z|y), p(y|z)$)

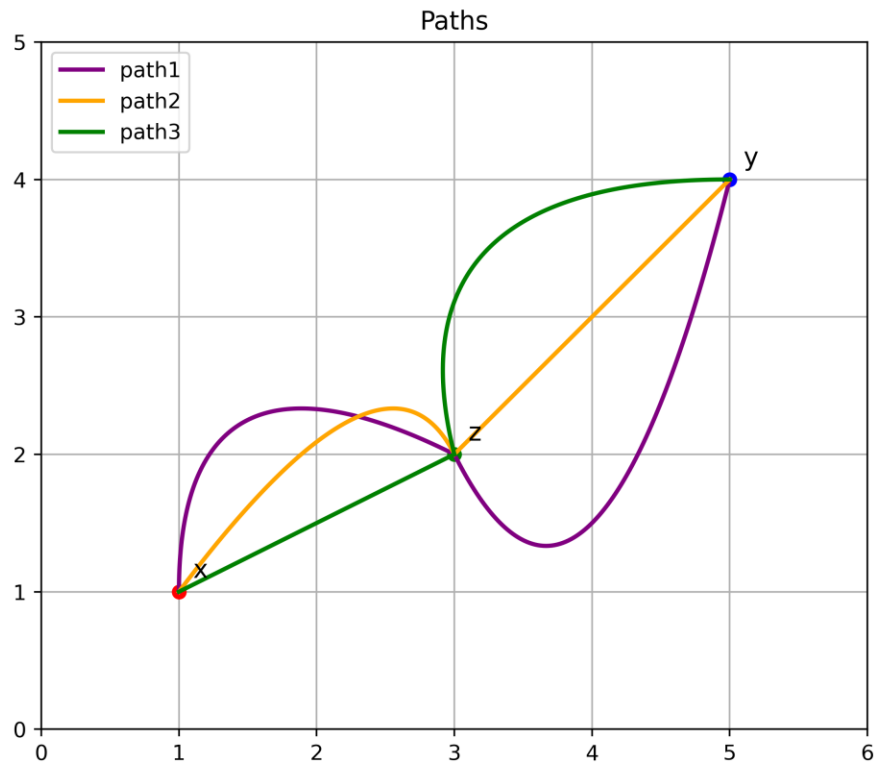
通常のニューラルネットワークは、 $x \rightarrow y$ の一方だけだけを end-to-end で学習します。しかし本研究では、逆方向の経路 (inverse path) も取り入れることで、「 z を通じた双方向の構造」を再構成できないかを探ります。

たとえば、 $x \rightarrow z$ と $y \rightarrow z$ の経路で同じ z にたどり着けるか、あるいは、 $z \rightarrow y$ の順方向に対して、逆方向の $y \rightarrow z \rightarrow x$ がどれくらい正しく再現できるかを検証します。

つまり、従来の end-to-end 学習のように $x \rightarrow y$ を「ひとつの関数」として学ぶのではなく、間に latent z をはさむことで、複数の経路の組み合わせによる柔軟な構造を考えようというのが本研究の考え方です。

このようにして、decoder の解釈性を高め、latent の持つ「構造的な意味」をより明らかにすることが、本研究の目的のひとつです。

考えを図式にする

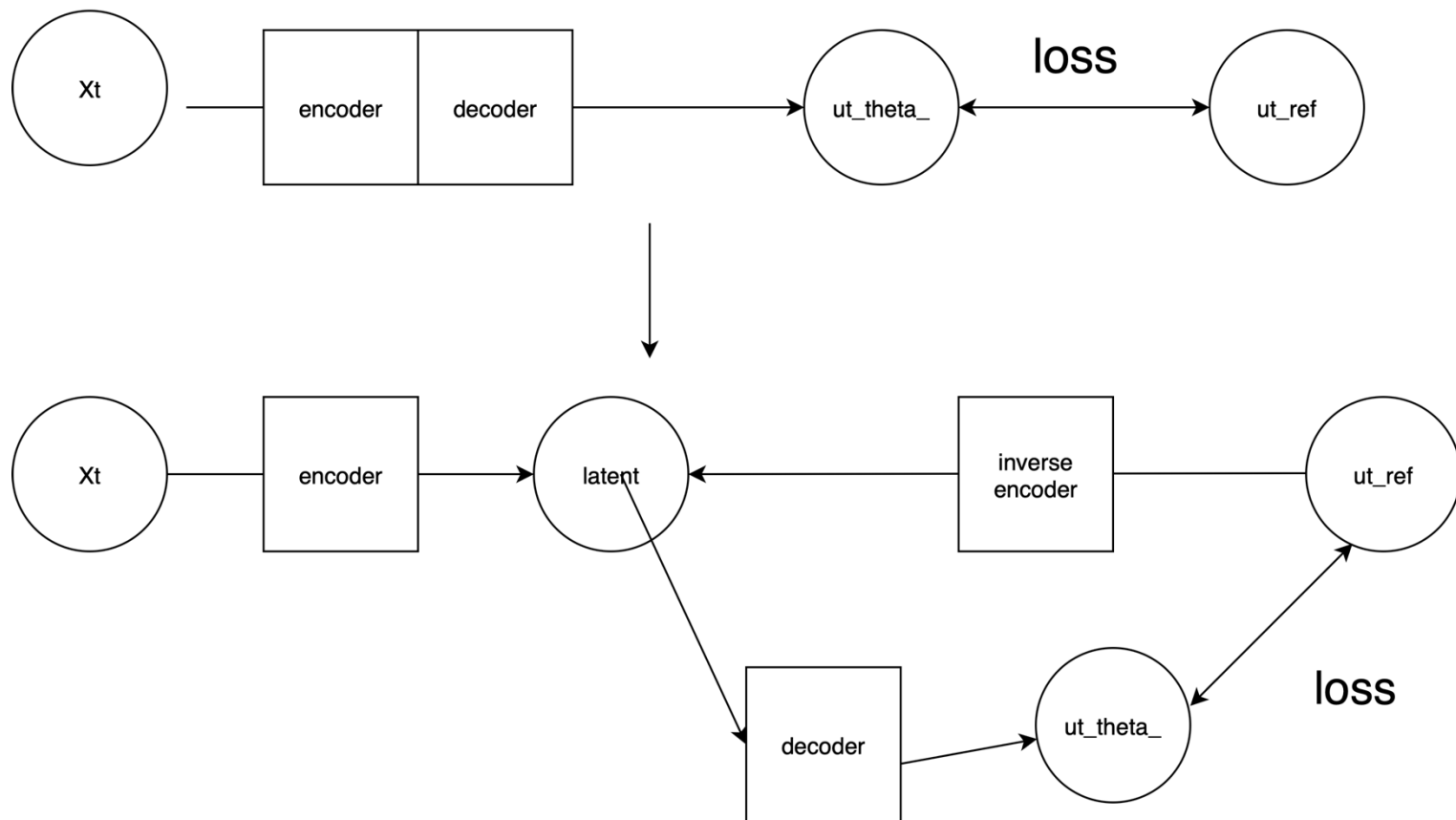


図に示すように、 x から y への経路は、 z を経由することで無数に存在する。
もし z をうまく見つけることができれば、 $x \rightarrow z \rightarrow y$ のように、**経路を分けて考える**ことができる。
つまり、 $x \rightarrow y$ を一発で学習するのではなく、「途中に z を挟む」ことで、**いろいろな組み合わせの path** が考えられるようになる。
この考え方のポイントは、「 z を経由する経路の組み合わせ」が自由に選べること。
隠れた状態・変数・空間 (=latent) を見つけければ、より柔軟で解釈しやすいモデルが作れる。

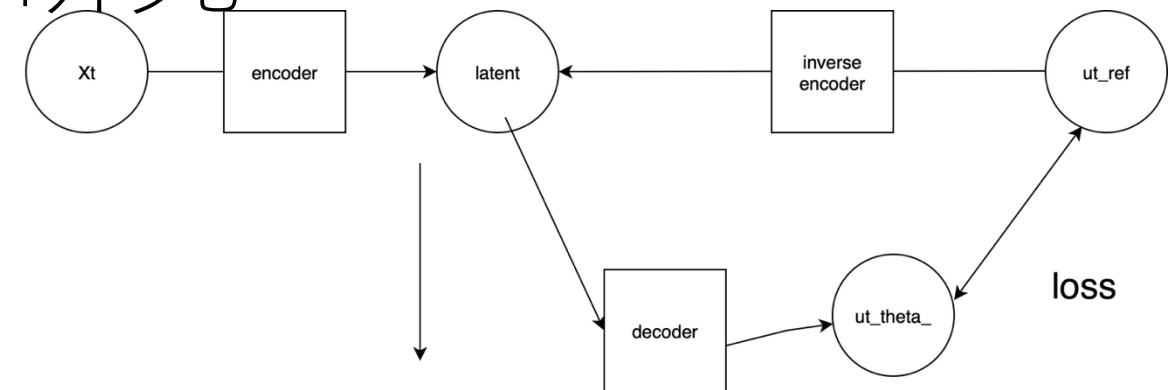
研究

図にあるように、従来の flow matching は上の構造で、自分が考えた方法は下の構造になる。encoder と decoder のあいだに latent という中間の隠れ変数を置いて、さらに inverse encoder も追加している。inverse encoder は ut_ref から latent にマッピングし、encoder は x_t から latent にマッピングする。このやり方で latent を明示的に表せるようになった。さらに、その latent を使って decoder で出力をつくり、 u_ref （正解）に近づくことを目指している。latent を通すことで、 x_t 側と ut 側の処理をほぼ独立に扱えるようになった。

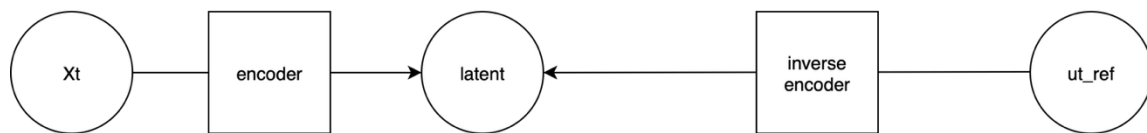
flow matching



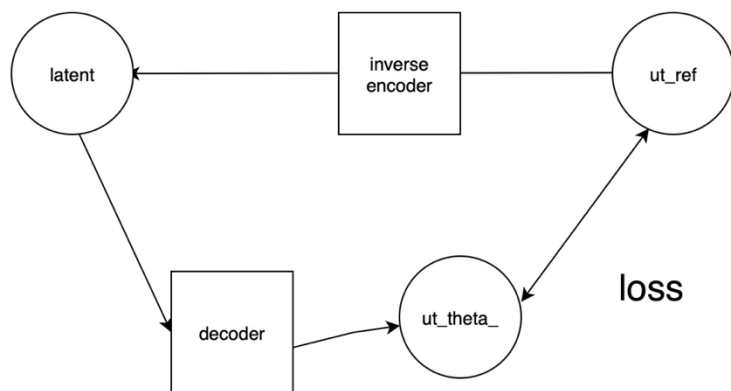
研究



part1



part2



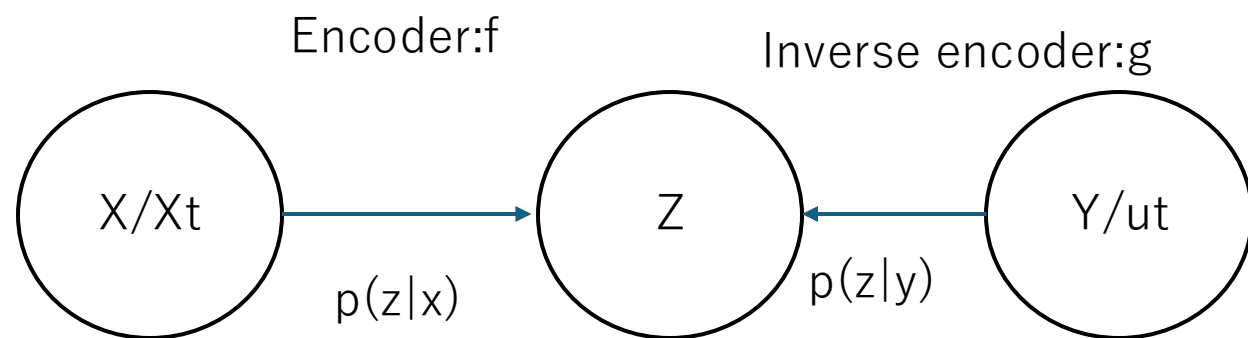
元の構造は、大きく 2 つの部分に分けることができる。

Part 1 では、Encoder と Inverse Encoder の出力差を比較することで、両者が同一の latent に到達できているかを検証できる。

Part 2 では、Inverse Encoder が ut_ref から latent にマッピングできれば、その latent を用いて Decoder が ut_theta を通じて ut_ref を再構成（近似）するようになる。

このようにして、出力が再び入力に戻る リサイクル構造 が形成される。

Latent



図式に示されるように、 $x \rightarrow z \leftarrow y$ の構造は、有向非巡回グラフ (DAG) における「コライダー」に相当する。 x から z への経路は $p(z|x)$ 、 y から z への経路は $p(z|y)$ と表現される。重要なのは、これら二つの経路が**同じ latent z に到達できるかどうか**である。同一の z にたどり着ける場合、 z を介して x と y の相互変換が可能となり、 z は単なる中間特徴ではなく、 **x と y を構造的に結びつける変数**として再解釈できる。

本研究では、 x からの encoder ($f(x)$) と y からの逆方向 encoder ($g(y)$) を同時に学習し、**最終的に $f(x) \doteq g(y)$** を満たすことを目指す。これにより、両方向から z へと写像した際に、一貫した latent 表現を得ることができる。

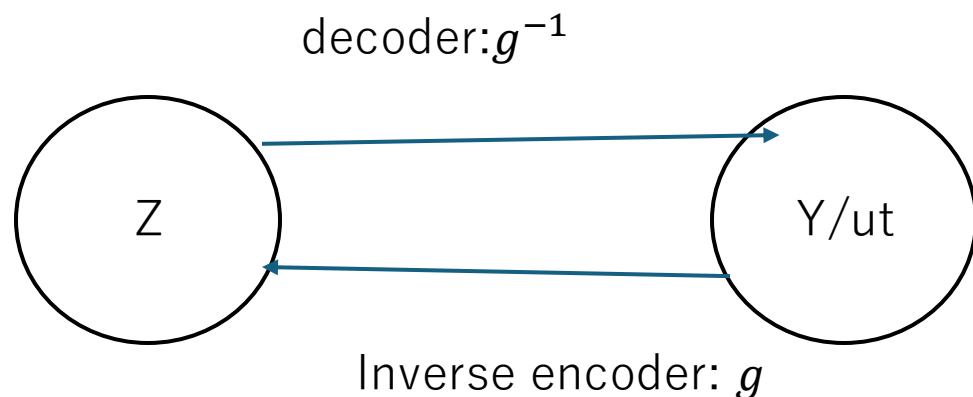
このときの損失関数は、たとえば

$$E(\|f(x) - g(y)\|^2)$$

のように定義される。実装では以下のように計算される：

```
einsum((ut_theta_latent - ut_ref_latent)**2, 'b seq d -> b')
```


inverse



$$g^{-1}(z) = g^{-1}(g(Y)) = Y$$

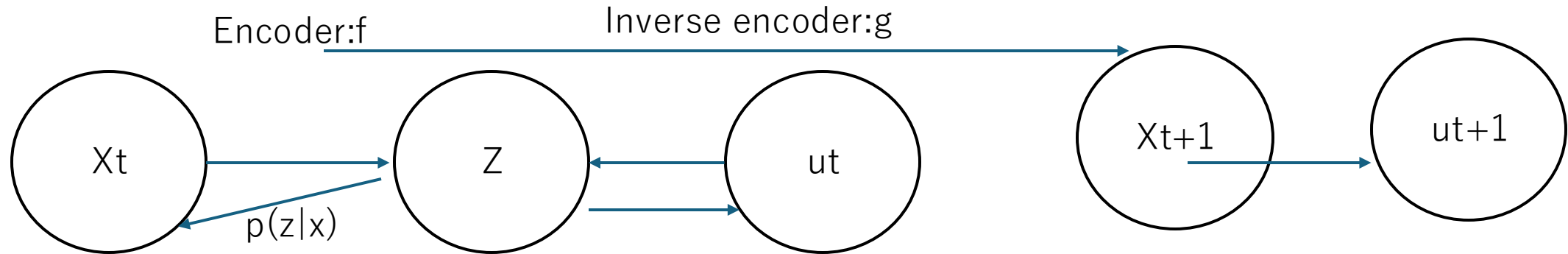
この構造を用いることで、inverse encoder を通じて decoder をトレーニングすることが可能となる。すなわち、隠れ変数 z と観測データ Y に基づいて decoder を学習させる構成である。

損失関数は次のように定義できる： $E(\|g^{-1}(g(Y)) - Y\|^2)$

実装上は $\text{einsum}((g^{-1}(g(Y)) - Y)^2, 'b c h w \rightarrow b')$ のような形で計算され、self-supervised な学習となる。

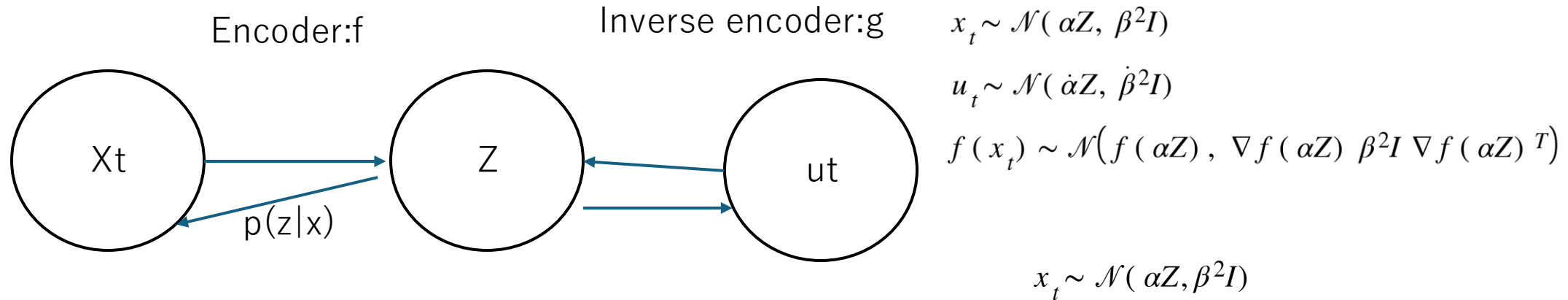
このように inverse encoder を活用することで decoder の学習を自己教師あり形式で行い、トレーニングの簡略化を目指す。

flow matchingとの繋がり(物理視点)



- Flow Matching は、ニューラルネットワークによって状態 X_t からベクトル場（速度） u_t への写像を学習する構造であり、これは物理的な意味を持つと考えられる。
ここで X_t は系の状態、 u_t はその変化（速度）として解釈でき、隠れ変数 z は運動エネルギー、モーメント、あるいは力学的ポテンシャルといった物理量として理解することが可能である。
ただし、加速度のような因果的関係を含む情報まで表現できるかどうかについては、依然として不明である。

flow matchingとの繋がり(統計的視点)



- 実験における Flow Matching では、 $x_t \sim \mathcal{N}(\alpha Z, \beta^2 I)$ 条件付き確率パスとして $x_t \sim \mathcal{N}(\alpha Z, \beta^2 * I)$ と仮定する。
- z はデータからサンプリング alpha beta は時間スケリング用
- ベクトルフィールド u_t は $\sim \mathcal{N}(\dot{\alpha} Z, \dot{\beta}^2 * I)$ になる
- delta methodにより $\mathcal{N}(f(\alpha Z), \nabla f(\alpha Z) \beta^2 I \nabla f^T(\alpha Z))$ f を通じた変換後もガウス分布が保たれる。measure transformationを通してガウス性が維持されることを意味する。

実験設計

- 本研究では以下の2つのアプローチにより、latent 表現の構造と再構成能力を評価する：
- **アプローチ1：**
 - Encoder と Inverse Encoder を同時に学習し、両者の出力が一致する latent を探索
 - Inverse Encoder をフリーズし、得られた latent と ground truth を用いて Decoder を学習
 - 最後に、Encoder + Decoder を組み合わせた fine-tuning を実施
- **アプローチ2：**
 - 先に Inverse Encoder と Decoder を同時に学習
 - Inverse Encoder を固定し、その上で Encoder を学習
 - 最後に全体を fine-tuning
- これらのアプローチは、latent の構造的意味の把握と decoder の解釈性向上を目的とする
- モデル選択は、以下の「重み付けメッセージ集計」の観点から多様な構造を比較する：
- **Attention (ViT)**：グローバルな重み付けによる情報集約（クラスタリング的構造）
- **Convolution (U-Net)**：ローカルな重み付けによる情報集約（局所構造重視）
- **MLP**：通常的全結合層による情報集約（線形近似的）
- **PCA + MLP**：PCA による基底展開と線形モデルによる情報抽出

実装方法

- コードの可読性向上のため、jaxtyping と einops を使用
- モジュール設計の柔軟性確保のため、ABC ライブラリを導入
 - Extractor(encoder)入力形式：
 - x: Float[Array, "batch channel height width"]
 - out: Float[Array, "batch seq dim"]
 - Optimizer : Adam, lr=1e-3
 - Random Seed : 42
 - 初期化はTruncated normal distributionで初期化
 - Decoder 入力形式:
 - x: Float[Array, " batch seq dim "]
 - out:Float[Array, " batch channel height width"]

実装方法

- **時間埋め込み**：Fourier Encoder を使用
- **ラベル埋め込み**：Embedding 層を使用
- 実験用に基本クラス Dense を定義し、カスタム線形層を実
- モデルは簡易構成とするため、縮小版の ViT/U-Net Encoder、MLP、および PCA+MLP により構築
- 間条件付きモデルのため、Context のようなクラスを定義し、
y と label を加算して latent を出力
(z: Float[Array, "batch seq dim"]、ここで $\text{seq} = h \times w$)

モデル	構造	活性化関数	output
MiniViT	Conv2d → +[CLS] Token + PosEmbedding → MultiHeadAttention (RotaryEmbedding) → Add & RMSNorm → FeedForward (Linear → GELU → Linear) → Add & RMSNorm → Linear Projection	GELU	Float [Array, B seq d]
MiniUNet	Conv2d → BatchNorm + SiLU → ResidualLayer (Conv2d × 2 + SiLU + BN + Add) → Conv2d (stride=2) → Rearrange to [B, Seq, Dim]	SiLU	Float [Array, B seq d]
MLP	Flatten → Linear → ReLU → Linear → Unsqueeze → [B, 1, d_out]	ReLU	Float [Array, B seq d]
PCAMLP	Flatten → PCA (fit from batch, d_in → n_components) → Linear → ReLU → Linear → Unsqueeze → [B, 1, d_out]	ReLU	Float [Array, B seq d]

モデル	構造	output
Utdecoder	x (latent vector) $\rightarrow$ einsum (x , y_embed) $\rightarrow$ einsum (x , t_embed) $\rightarrow$ Rearrange $\rightarrow$ ConvTranspose2d $\rightarrow$ Upsample $\rightarrow$ GELU $\rightarrow$ Conv2d $\rightarrow$ BN $\rightarrow$ Conv2d	Float [Array, B c h w]
Matcher	$x \rightarrow$ latent_model $\rightarrow t \rightarrow$ FourierEmbed(t) $y \rightarrow$ Embedding(y) $\rightarrow$ einsum (x , y_embed) $\rightarrow$ einsum (x , t_embed) $\rightarrow$ Rearrange $\rightarrow$ ConvTranspose2d $\rightarrow$ Upsample $\rightarrow$ GELU $\rightarrow$ Conv2d $\rightarrow$ BN $\rightarrow$ Conv2d	Float [Array, B c h w]
Contexter	$x \rightarrow$ latent_model $\rightarrow t \rightarrow$ FourierEmbed(t), $y \rightarrow$ Embedding(y) $\rightarrow$ einsum (x , y_embed) $\rightarrow$ einsum (x , t_embed)	Float [Array, B seq d]
LatentVecField	$x, t, y \rightarrow$ Contexter	Float [Array, B, Seq, D]
vecfield	encoder $\rightarrow$ decoder	Float[Array, B c h w]

実験

実験	内容
実験1:End to end 実験1	Encoder（一部マスクあり） + matcher (t, y なし) による End-to-End トレーニング（初期化から実施）
実験2:End to end 実験2	Encoder + utdecoder (t, y あり) による End-to-End トレーニング（初期化から実施）
実験3:Latent実験1	Encoder と Inverse Encoder を同時にトレーニング（初期化から実施）
実験4:inverse実験1	実験3で得られた Inverse Encoder を固定し、Decoder をトレーニング （Decoder のみ初期化）
実験5: combine 実験1	実験3および4で学習した Encoder と Decoder を用いて End-to-End トレーニング

実験

実験	内容
実験6:combnine 実験2	Latent 実験① と Inverse 実験① で得られた Encoder・Inverse Encoder・Decoder を同時にトレーニング
実験7:Full 実験1	Encoder・Inverse Encoder・Decoder をすべて初期化し、最初から同時にトレーニングを実施
実験8:inverse実験2	Inverse Encoder と Decoder を初期化し、両者を同時にトレーニング
実験9:Latent実験2	Encoder を初期化し、事前学習済みの Inverse Encoder を固定した上で Encoder のみをトレーニング
実験10:combnine 実験3	実験8・9 で得られた Encoder と Decoder を用いて End-to-End トレーニングを実施

実験モデルとパラメータとサイズ

Model	Forward Params	Forward Size	Inverse Params	Inverse Size	Decoder Params	Decoder Size
MiniViT	16008	0.06 MiB	15824	0.06 MiB	17177	0.07 MiB
ConvNet	16580	0.06 MiB	16396	0.06 MiB	17177	0.07 MiB
PCA_MLP	15761	1.06 MiB	15850	1.06 MiB	17177	0.07 MiB
MLP	16799	0.06 MiB	16638	0.06 MiB	15106	0.06 MiB

実験2～11では、すべて同一のパラメータ数で実験を行っている。
Decoder はすべて共通の構造を採用しており、スライドに記載した utdecoder を使用している。

また、Inverse Encoder については、時間 t およびラベル y を含まない構造で、

Encoder と同じアーキテクチャを使用している。

実験結果の評価方法と結論

- **実験結果の評価方法と結論**
- 評価には以下の指標を使用：
 - Loss 関数による損失
 - Cos 類似度による近似性と方向の一致
 - KL 散度による順・逆経路の違いの定量化
- 発見
- Encoder・Decoder・Inverse Encoder を同時に学習させた場合、**loss の低下が不十分だった**
- 「先に latent を学習 → その latent と Inverse Encoder を用いて Decoder を学習」する方法は、
「先に Inverse Encoder と Decoder を学習 → 後から Encoder を学習する」方法に比べ
loss が大きくなる傾向があった（特に ViT は latent を異なる方向に学習することがあった）
- 最初に Decoder を学習し、その後 Encoder・Decoder を合わせて再学習（End-to-End）する方法は、
最初から End-to-End で学習した場合に近い性能が得られた

結論

- **Decoder** を先に学習させる構成においては、**Decoder** の **loss** が低いモデルほど、その後学習する **Encoder** の **loss** は高くなる傾向があった。
結果的に平均**loss**は普通の**end-to-end**より低い**loss**ができた
- **T, t pair ks test**を行うと **vit**と**mlp**は**t**と**ks**で**fail to reject**が出た。この結果から見れば逆の**path**からは完全に**end-to-end**に至るその上結果がよいになるのも可能。
- **latent** における **loss** が小さいことは望ましいが、それ以上に **Decoder** の質が重要であり、
解釈可能性の高い **Decoder** を設計することが鍵となる。
良い表現 (**representation**) だけでなく、それを正しく活用・出力できる **Decoder** の設計も必要である。
特に **self-supervised learning** においては **Transformer** 系のモデルが一番

実験結果1~12

	Model	Type	Loss	CosineSimilarity	KL(x y)	KL(y x)
0	MiniViT	path1_latent	1.087391	-0.000542	261.990784	3.525642e+03
1	ConvNet	path1_latent	0.052242	0.011837	44.572266	5.013416e+01
2	PCA_MLP	path1_latent	0.952653	0.010106	10.969122	1.807810e+01
3	MLP	path1_latent	0.910717	-0.065822	30.755337	3.839958e+03
4	MiniViT	path1_latent	1.087391	-0.000542	261.990784	3.525642e+03
5	ConvNet	path1_latent	0.052242	0.011837	44.572266	5.013416e+01
6	PCA_MLP	path1_latent	0.952653	0.010106	10.969122	1.807810e+01
7	MLP	path1_latent	0.910717	-0.065822	30.755337	3.839958e+03
8	MiniViT	path2_latent	1961.757446	0.609404	15066.899414	1.134999e+03
9	ConvNet	path2_latent	2150.946289	0.169265	369647.250000	2.380992e+05
10	PCA_MLP	path2_latent	54.453049	0.165267	69.507935	8.780203e+00
11	MLP	path2_latent	16.300381	0.404308	34.257347	4.519922e+00

結果 13~28

12 MiniViT	path1_inverse	1300.916992	0.569857	3762.792969	5.016171e+06
13 ConvNet	path1_inverse	1363.395874	0.540587	3842.021484	5.142098e+06
14 PCA_MLP	path1_inverse	1347.551758	0.548863	4086.049561	6.022640e+06
15 MLP	path1_inverse	1280.134277	0.579587	4096.172852	6.399210e+06
16 MiniViT	path2_inverse	1147.877808	0.635328	3107.946045	3.229647e+06
17 ConvNet	path2_inverse	1195.697754	0.615374	2810.270264	2.499337e+06
18 PCA_MLP	path2_inverse	1523.263306	0.505149	3641.780762	4.103128e+06
19 MLP	path2_inverse	1247.351440	0.593808	3936.652344	4.979682e+06
20 MiniViT	path1_combine	1254.539795	0.590444	3728.073730	4.865774e+06
21 MiniUNet	path1_combine	1215.498169	0.607196	3442.209229	4.090250e+06
22 PCAMLP	path1_combine	1251.458496	0.592249	4029.672852	5.678129e+06
23 MLP	path1_combine	1275.135864	0.583827	4065.090820	5.992030e+06
24 MiniViT	path2_combine	1128.647095	0.643063	3043.034424	3.026630e+06
25 MiniUNet	path2_combine	1134.848999	0.640581	2505.318604	2.068081e+06
26 PCAMLP	path2_combine	1256.227905	0.590112	3772.582764	4.445696e+06
27 MLP	path2_combine	1161.623413	0.629732	3720.623047	4.268304e+06

結果29~44

32	MiniViT	combine	1421.605591	0.520440	3740.140625	5.098218e+06
33	MiniViT	combine	1316.710693	0.566853	3506.402832	4.258320e+06
34	MiniViT	combine	159.577316	0.863083	16566.837891	5.862900e+02
35	UNet	combine	1252.170410	0.591413	3726.481689	5.031862e+06
36	UNet	combine	1224.985352	0.603094	3693.037109	4.995496e+06
37	UNet	combine	15.680247	0.988432	72.135437	5.175476e+01
38	PCA_MLP	combine	1352.504395	0.559236	4074.872803	5.971206e+06
39	PCA_MLP	combine	1273.071533	0.583455	4038.970703	5.856498e+06
40	PCA_MLP	combine	88.598854	0.953462	348.896240	6.024109e+01
41	MLP	combine	1387.692749	0.564080	4137.935059	6.054228e+06
42	MLP	combine	1276.392700	0.582055	4096.803711	6.380503e+06
43	MLP	combine	110.316612	0.970927	28.393097	5.518257e+05

28	MiniViT	path0	1136.645630	0.640099	3100.699219	2.893426e+06
29	MiniUNet	path0	1177.398193	0.623277	3073.382324	3.030573e+06
30	PCAMLP	path0	1250.250977	0.593740	3734.596680	4.263134e+06
31	MLP	path0	1172.771851	0.625085	3726.212891	4.269414e+06

結果45~56

44	MiniViT	CM	1904.59069 8	0.439094	4569.58349 6	6.481372e+ 06
45	MiniViT	CM	1881.62597 7	0.530947	4559.34668 0	7.850358e+ 06
46	MiniViT	CM	0.294458	0.902379	158.862610	7.550081e+ 02
47	UNet	CM	1882.80847 2	0.222496	4581.18408 2	6.842779e+ 06
48	UNet	CM	1878.74072 3	0.242329	4577.98632 8	7.478447e+ 06
49	UNet	CM	0.022200	0.998521	21.068542	5.439160e+ 01
50	PCA_MLP	CM	1904.24548 3	0.260037	4500.31543 0	5.589407e+ 06
51	PCA_MLP	CM	1578.95959 5	0.442376	4288.50097 7	6.384272e+ 06
52	PCA_MLP	CM	185.918365	0.769015	93.670410	9.294925e+ 04
53	MLP	CM	1427.79480 0	0.509234	4177.33203 1	6.317109e+ 06
54	MLP	CM	1415.00903 3	0.513054	4191.94677 7	6.762758e+ 06
55	MLP	CM	8.590675	0.997838	31.039429	2.761283e+ 03